GP-6 · Chapters 5, 6 & 9 · Security-Focused Development Project

Build Secure. Implement Security Controls in a Simple Application

SCENARIO:

A student entrepreneurship team at AAU is building a community resource-sharing platform. Users can register, log in, and post study resources. They have asked your group to build a minimal, secure prototype that demonstrates the security controls from Chapters 5, 6, and 9. Feature richness is not the priority of security controls is.

Project	**GP-6: Build Secure: Security-Focused Application**
Total Marks	**20 Marks**
Group Size	5–8 students
Duration	2 Weeks
Deliverables	Working app + security design document 800–1,000 words + code review report 500–700 words + 10-minute live demo
Submission	Code (ZIP) + PDF documents via Google Classroom 12 hours before demo day
Scope	Registration + login + post submission. Focus on the 6 security controls — not on features.

Phases

This project runs over 2 weeks. Stay on schedule.

PHASE 1: Security Design: Before Any Code

- Threat model: 4 threats using STRIDE. Table: Threat | STRIDE Category | Control You Will Implement.
- Input validation plan: for each user input (username, email, password, post body) allowed characters, max length, server-side validation rule.
- Authentication design: password hashing algorithm (must be bcrypt or Argon2, state the specific library and work factor), session token approach, cookie flags.

PHASE 2: Secure Implementation ·

Implement ALL 6 required security controls from the start, not added later:

- Passwords hashed with bcrypt or Argon2 (NOT MD5 or plain SHA-256)
- All database queries use parameterized statements — NO string concatenation
- All user output HTML-escaped before rendering
- Session cookies have Http Only and Secure flags
- 3+ HTTP security headers on every response (CSP · X-Content-Type-Options · X-Frame-Options)

- Login rate limiting: 5 failed attempts → 15-minute lockout

PHASE 3 Self Code Review

a. SQL injection check: show the specific parameterized query line from your code.
b. XSS check: show the specific output-escaping function call.
c. Authentication check: show the bcrypt/Argon2 call at registration and at login.
d. Rate limiting test: document what happens at the 6th consecutive failed login.
e. Self-identified weakness: one control you could not fully implement. state the risk and the specific fix.

PHASE 4: Live Demonstration

10 minutes. Demonstrate all 6 security controls live:

- Attempt SQL injection in search (show blocked)
- Submit XSS in post body (show escaped in output)
- 6 rapid failed logins (show lockout activates)
- Browser dev tools showing cookie flags (Http Only, Secure) and 3 security headers
- Show bcrypt hash in database
- Present self-identified weakness with the specific fix
- All members speak.

Deliverables

- Working app
- security design document 800–1,000 words
- code review report 500–700 words
- 10-minute live demo

Presentation Guidance

- All group members must speak — any member may be questioned during Q&A.
- Use diagrams and tables on slides rather than paragraphs of text.
- Prepare a backup plan (screenshots) for any live demonstrations.
- • Each member completes a confidential peer evaluation form after the presentation.

Marking Rubric

Written work and presentation are both assessed. A report that cannot be explained or defended during the presentation will receive a lower grade.

Criterion	Marks	Excellent 90–100%	Good 70–89%	Satisfactory 50–69%	Poor < 50%
Security Design	4	STRIDE model complete (4 threats); input validation	3–4 STRIDE threats; most fields in validation	2–3 threats; partial validation plan;	No STRIDE; no validation plan; no

Criterion	Marks	Excellent 90–100%	Good 70–89%	Satisfactory 50–69%	Poor < 50%
		covers all fields with specific rules; authentication design specifies algorithm with correct justification and all cookie flags.	plan; authentication mostly complete.	authentication partially defined.	authentication design.
Implementation	**8**	All 6 controls correctly implemented and verifiable in code; no SQLi or XSS vulnerabilities in working code; rate limiting functions correctly.	5 of 6 controls; no critical vulnerabilities; rate limiting mostly works.	3–4 controls; minor vulnerabilities; rate limiting partial.	Fewer than 3 controls; critical vulnerabilities present; no rate limiting.
Code Review Report	**4**	All 5 review areas with specific line references; rate limiting test documented; self-identified weakness genuine and correctly assessed with specific fix.	4–5 areas with references; weakness identified with fix.	2–3 areas; partial references; weakness vague.	Fewer than 2 areas; no references; no weakness.
Live Demo	**4**	All 6 controls demonstrated successfully; SQLi and XSS correctly prevented; cookie flags	4–5 controls demonstrated; most correct; weakness presented; most members speak.	2–3 controls; some correct; weakness vague; some members speak.	Fewer than 2 controls; demonstrations incorrect; no weakness.

Criterion	Marks	Excellent 90–100%	Good 70–89%	Satisfactory 50–69%	Poor < 50%
		and headers visible; weakness presented with correct fix; all members speak.			
TOTAL	**20**	Partial marks are awarded within each band. Precision and correct application of course concepts are rewarded, not length.			